CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology and Engineering

Devang Patel Institute of Advance Technology and Research

Department of Information Technology

BIG DATA ANALYTICS (CSUE301)

PRACTICAL 5

Perform CRUD Operations Using MongoDB

24DIT021

1. OBJECTIVE

The objective of this practical is to work with MongoDB, a NoSQL document database, and perform basic database operations on document-oriented data. The practical covers database and collection creation, document insertion, querying, updating, deletion and aggregation. It also introduces indexing and JSON data import for simple analytical work.

2. AIM

To install and configure MongoDB, create a database and collection, perform Create, Read, Update and Delete operations using MongoDB Shell, and use the aggregation pipeline to analyse student records.

3. PROBLEM DEFINITION

A large amount of semi-structured data cannot always be conveniently represented using fixed relational tables. In this practical, student records are stored as MongoDB documents. The database is used to insert, retrieve, modify and remove documents and to perform simple analysis using aggregation. The practical demonstrates how a document database can manage flexible data structures in a Big Data environment.

4. CO / PO MAPPING

This practical is mapped with CO2. The practical develops the ability to work with a NoSQL database, manage document-based data and perform basic data analysis using MongoDB operations.

5. TOOLS / TECHNOLOGY USED

- Ubuntu Linux / WSL Terminal
- MongoDB Community Server 8.2.3 (Windows)
- MongoDB Shell (mongosh) 2.5.8
- MongoDB Compass
- VS Code
- JSON dataset

6. WORKING FOLDER

A separate folder was used for Practical 5. The practical files and dataset were kept in the following folder so that earlier practicals remained unchanged.

```
C:\Users\Hit\Desktop\5th SEM\BDA\Practical\Practical_05_MongoDB_CRUD_Operations
WSL path: /mnt/c/Users/Hit/Desktop/5th SEM/BDA/Practical/Practical_05_MongoDB_CRUD_Operations
pwd
ls -lah
```

7. MONGODB INSTALLATION AND CONFIGURATION

MongoDB Community Server 8.2.3 is installed natively on Windows and runs as the Windows MongoDB service. MongoDB Shell (mongosh) 2.5.8 is installed in WSL. WSL mirrored networking was enabled so that the Windows MongoDB server can be accessed from WSL through 127.0.0.1:27017.

```
Windows PowerShell: Get-Service MongoDB
Windows PowerShell: netstat -ano | findstr :27017
mongosh "mongodb://127.0.0.1:27017"
```

The MongoDB connection was verified from WSL using the local Windows host address. A successful ping confirmed communication between mongosh and the MongoDB server.

```
db.version()
db.adminCommand({ ping: 1 })
```

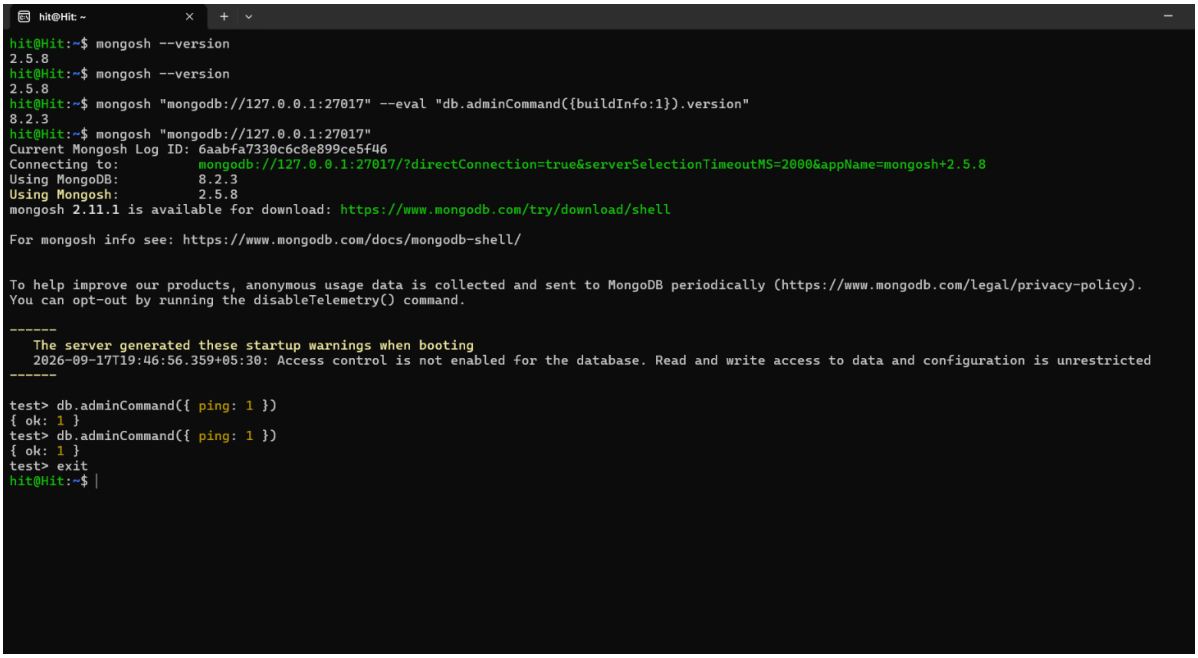


Figure 1: MongoDB and mongosh version verification and successful ping.

The successful ping response was { ok: 1 }. The server version reported by the connected MongoDB instance was 8.2.3.

8. CREATING DATABASE AND COLLECTION

A database named BDA_Practical5 was used for this practical. The students collection was created inside this database.

```
use BDA_Practical5
db.createCollection("students")
```

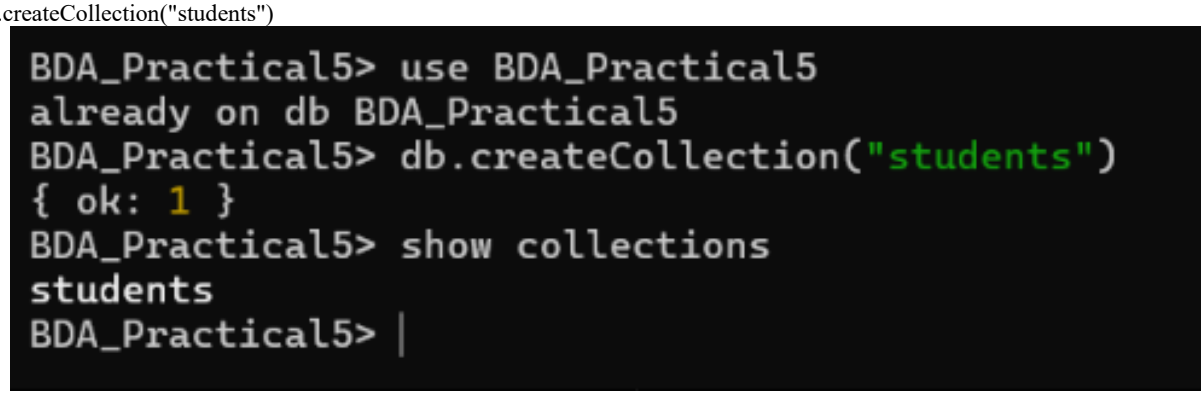


Figure 2: Database selection, students collection creation and collection verification.

```
show dbs
show collections
```

The students collection is used to store student documents containing fields such as rollNo, name, department, semester and marks.

9. SAMPLE TEST DATA

The following student documents were used as sample test data. MongoDB stores these documents internally in BSON format.

```
{
 rollNo: 101,
 name: "Rahul",
 department: "IT",
 semester: 5,
 age: 20,
 city: "Surat",
 marks: 82
}
{
 rollNo: 102,
 name: "Priya",
 department: "CE",
 semester: 5,
 age: 21,
 city: "Vadodara",
 marks: 91
}
{
 rollNo: 103,
 name: "Amit",
 department: "IT",
 semester: 5,
 age: 20,
 city: "Ahmedabad",
 marks: 74
}
{
 rollNo: 104,
 name: "Neha",
 department: "CSE",
 semester: 5,
 age: 22,
 city: "Surat",
 marks: 88
}
{
 rollNo: 105,
 name: "Karan",
 department: "IT",
 semester: 5,
 age: 21,
 city: "Rajkot",
 marks: 68
}
```

10. CREATE OPERATION - INSERT DOCUMENTS

Create operation is performed by inserting documents into the collection. MongoDB provides insertOne() for a single document and insertMany() for multiple documents.

```
db.students.insertOne({
 rollNo: 101,
 name: "Rahul",
 department: "IT",
 semester: 5,
```

```
  age: 20,
  city: "Surat",
  marks: 82
})
```

```
BDA_Practical5> db.students.insertOne({
...   rollNo: 101,
...   name: "Rahul",
...   department: "IT",
...   semester: 5,
...   age: 20,
...   city: "Surat",
...   marks: 82
... })
{
  acknowledged: true,
  insertedId: ObjectId('6aabfaf9404caf6aa6ce5f47')
}
BDA_Practical5> db.students.insertMany([
```

Figure 3: Create operation using insertOne().

Multiple documents can be inserted using insertMany().

```
db.students.insertMany([
  {rollNo: 102, name: "Priya", department: "CE", semester: 5, age: 21, city: "Vadodara", marks: 91},
  {rollNo: 103, name: "Amit", department: "IT", semester: 5, age: 20, city: "Ahmedabad", marks: 74},
  {rollNo: 104, name: "Neha", department: "CSE", semester: 5, age: 22, city: "Surat", marks: 88},
  {rollNo: 105, name: "Karan", department: "IT", semester: 5, age: 21, city: "Rajkot", marks: 68},
  {rollNo: 106, name: "Pooja", department: "CE", semester: 5, age: 20, city: "Surat", marks: 79},
  {rollNo: 107, name: "Vivek", department: "CSE", semester: 5, age: 22, city: "Ahmedabad", marks: 93}
])
```

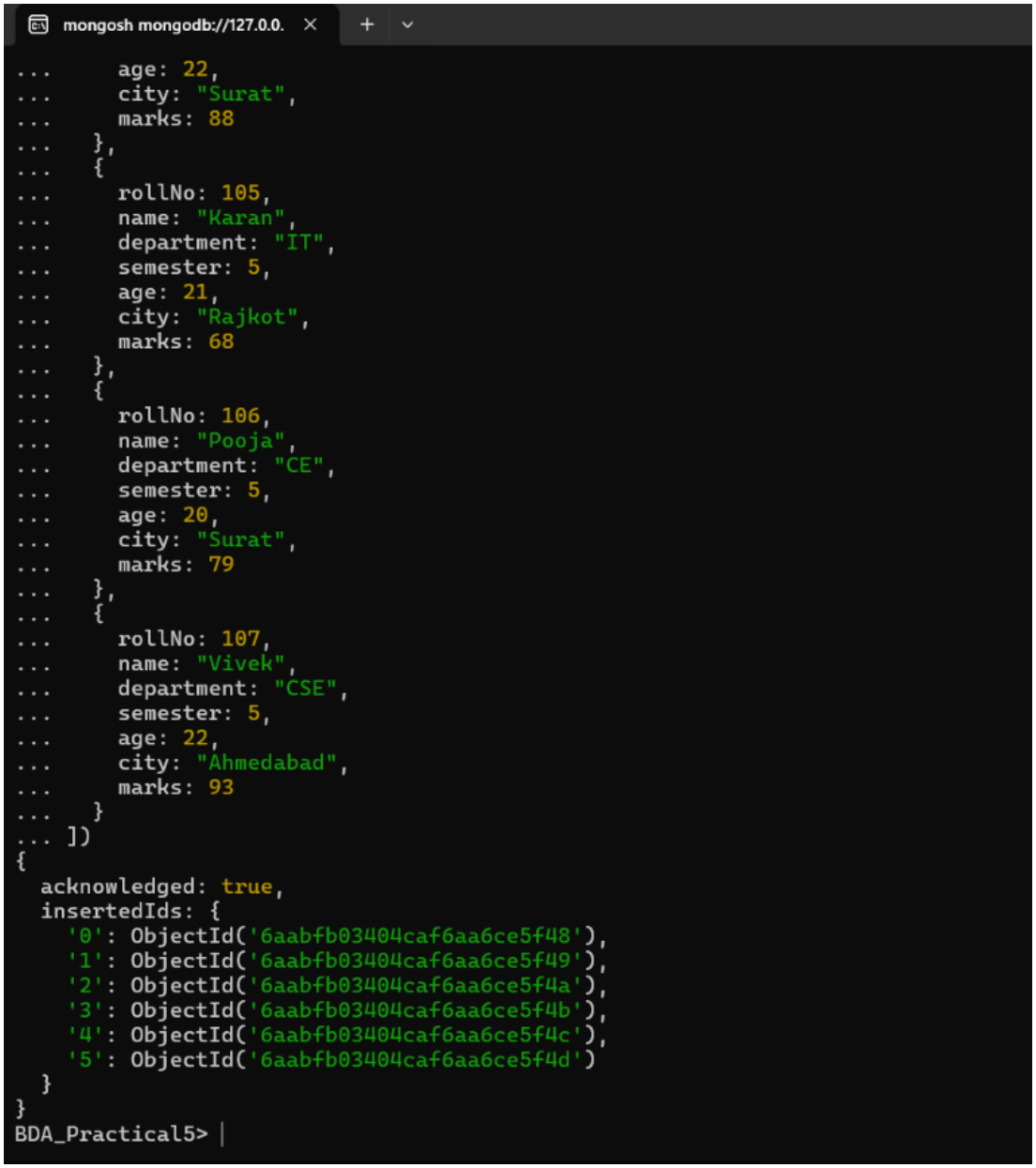


Figure 4: Create operation using insertMany().

db.students.countDocuments()

11. READ OPERATION - QUERY DOCUMENTS

Read operation is used to retrieve documents from a collection. The find() method returns documents matching the specified condition.

db.students.find({}, {_id: 0}).forEach(printjson)

```
BDA_Practical5> db.students.find({}, {_id:0}).forEach(printjson)
{
  rollNo: 101,
  name: 'Rahul',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Surat',
  marks: 87,
  course: 'Big Data Analytics'
}
{
  rollNo: 102,
  name: 'Priya',
  department: 'CE',
  semester: 5,
  age: 21,
  city: 'Vadodara',
  marks: 91
}
{
  rollNo: 103,
  name: 'Amit',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Ahmedabad',
  marks: 74,
  course: 'Big Data Analytics'
}
{
  rollNo: 104,
  name: 'Neha',
  department: 'CSE',
  semester: 5,
  age: 22,
  city: 'Surat',
  marks: 88
}
{
  rollNo: 105,
  name: 'Karan',
```

Figure 5: Read operation displaying student documents.

db.students.find({department: "IT"}, {_id: 0}).forEach(printjson)

```
BDA_Practical5> db.students.find({department:"IT"}, {_id:0}).forEach(printjson)
{
  rollNo: 101,
  name: 'Rahul',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Surat',
  marks: 87,
  course: 'Big Data Analytics'
}
{
  rollNo: 103,
  name: 'Amit',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Ahmedabad',
  marks: 74,
  course: 'Big Data Analytics'
}
{
  rollNo: 105,
  name: 'Karan',
  department: 'IT',
  semester: 5,
  age: 21,
  city: 'Rajkot',
  marks: 73,
  course: 'Big Data Analytics'
}
{
  rollNo: 101,
  name: 'Rahul',
  department: 'IT',
```

Figure 6: Read operation using a department filter.

db.students.find({marks: {$gt: 80}}, {_id: 0}).forEach(printjson)

```
BDA_Practical5> db.students.find({marks:{$gt:80}}, {_id:0}).forEach(printjson)
{
  rollNo: 101,
  name: 'Rahul',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Surat',
  marks: 87,
  course: 'Big Data Analytics'
}
{
  rollNo: 102,
  name: 'Priya',
  department: 'CE',
  semester: 5,
  age: 21,
  city: 'Vadodara',
  marks: 91
}
{
  rollNo: 104,
  name: 'Neha',
  department: 'CSE',
  semester: 5,
  age: 22,
  city: 'Surat',
  marks: 88
}
{
  rollNo: 106,
  name: 'Anjali',
```

Figure 7: Read operation using a marks filter.

```
db.students.find({semester: 5, department: "IT"}, {_id: 0}).forEach(printjson)
```

The findOne() method can be used when only one matching document is required.

```
db.students.findOne({rollNo: 103})
```

12. QUERY USING PROJECTION AND SORTING

Projection is used to select only required fields from the result. The value 1 includes a field and the value 0 excludes a field.

```
db.students.find({}, {_id: 0, name: 1, department: 1, marks: 1}).forEach(printjson)
```

```
BDA_Practical5> db.students.find(
...     {},
...     {_id:0,name:1,department:1,marks:1}
... ).forEach(printjson)
{
  name: 'Rahul',
  department: 'IT',
  marks: 87
}
{
  name: 'Priya',
  department: 'CE',
  marks: 91
}
{
  name: 'Amit',
  department: 'IT',
  marks: 74
}
{
  name: 'Neha',
  department: 'CSE',
  marks: 88
}
{
  name: 'Karan',
  department: 'IT',
  marks: 73
}
{
  name: 'Anjali',
  department: 'CE',
```

Figure 8: Projection and sorting query results.

Documents can also be sorted according to a field. A value of 1 gives ascending order and -1 gives descending order.

```
db.students.find({}, {_id: 0, name: 1, marks: 1}).sort({marks: -1}).forEach(printjson)
db.students.find().sort({name: 1})
```

A query can also limit the number of documents returned.

```
db.students.find().sort({marks: -1}).limit(3)
```

13. UPDATE OPERATION

Update operation modifies an existing document. updateOne() updates the first document that matches the filter. The $set operator changes the specified field without replacing the complete document.

```
db.students.updateOne(
  {rollNo: 104},
  {$set: {marks: 72}}
)
```

```
BDA_Practical5> db.students.updateMany(
...    {marks:{$lt:75}},
...    {$inc:{marks:5}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
BDA_Practical5> db.students.find(
...    {},
...    {_id:0,name:1,marks:1}
... ).forEach(printjson)
{
  name: 'Rahul',
  marks: 87
}
{
  name: 'Priya',
  marks: 91
}
{
  name: 'Amit',
  marks: 79
}
{
  name: 'Neha',
  marks: 88
```

Figure 9: Update operation using updateMany() and verification of updated marks.

The updated document can be checked using find().

```
db.students.findOne({rollNo: 104})
```

Multiple documents can be updated using updateMany(). For example, the semester field can be changed for all documents that satisfy a condition.

```
db.students.updateMany(
  {department: "IT"},
  {$set: {course: "Big Data Analytics"}}
)
```

14. DELETE OPERATION

Delete operation removes documents from a collection. deleteOne() removes one matching document and deleteMany() removes all documents matching the given condition.

```
db.students.deleteOne({rollNo: 107})
```

```
BDA_Practical5> db.students.deleteOne({rollNo:107})
{ acknowledged: true, deletedCount: 1 }
BDA_Practical5> db.students.deleteMany({marks:{$lt:70}})
{ acknowledged: true, deletedCount: 0 }
BDA_Practical5> db.students.find(
...   {},
...   {_id:0}
... ).forEach(printjson)
{
  rollNo: 101,
  name: 'Rahul',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Surat',
  marks: 87,
  course: 'Big Data Analytics'
}
{
  rollNo: 102,
  name: 'Priya',
  department: 'CE',
  semester: 5,
  age: 21,
  city: 'Vadodara',
  marks: 91
}
{
  rollNo: 103,
  name: 'Amit',
  department: 'IT',
  semester: 5,
  age: 20,
  city: 'Ahmedabad',
  marks: 79,
  course: 'Big Data Analytics'
}
{
  rollNo: 104,
```

Figure 10: Delete operations and verification of remaining documents.

```
db.students.deleteMany({marks: {$lt: 70}})
```

The drop() method can remove a collection and its documents. For safety, the collection deletion demonstration was performed on a temporary collection rather than deleting the main students collection.

```
db.createCollection("temporaryCollection")
db.temporaryCollection.drop()
```

The drop operation permanently removes the selected collection from the current database.

15. BSON FORMAT

MongoDB stores documents in BSON, which stands for Binary JSON. BSON represents JSON-like documents in a binary form and supports additional data types. A MongoDB document can therefore contain fields with different data types, including strings, numbers, arrays, embedded documents and dates. BSON is used internally for storage and communication between MongoDB components.

16. AGGREGATION PIPELINE

The aggregation pipeline processes documents through a sequence of stages. Each stage receives documents, performs an operation and passes the resulting documents to the next stage. It is useful for filtering, grouping, sorting and calculating summary information.

The students collection was retained after CRUD operations so that aggregation and indexing could be demonstrated.

```
db.students.aggregate([
  {$match: {semester: 5}},
  {$group: {
    _id: "$department",
    averageMarks: {$avg: "$marks"},
    studentCount: {$sum: 1}
  }},
  {$sort: {averageMarks: -1}}
])
```

```
BDA_Practical5> db.students.aggregate([
...   {
...     $group:{
...       _id:"$department",
...       totalStudents:{$sum:1},
...       averageMarks:{$avg:"$marks"},
...       highestMarks:{$max:"$marks"},
...       lowestMarks:{$min:"$marks"}
...     }
...   },
...   {
...     $sort:{averageMarks:-1}
...   }
... ]).forEach(printjson)
{
  _id: 'CE',
  totalStudents: 4,
  averageMarks: 89,
  highestMarks: 95,
  lowestMarks: 79
}
{
  _id: 'CSE',
  totalStudents: 2,
  averageMarks: 88,
  highestMarks: 88,
  lowestMarks: 88
}
{
  _id: 'IT',
  totalStudents: 6,
  averageMarks: 79.66666666666667,
  highestMarks: 87,
  lowestMarks: 73
}

BDA_Practical5>
```

Figure 11: Aggregation pipeline producing department-wise summary statistics.

In this pipeline, $match selects fifth-semester students. $group creates one group for each department and calculates the average marks and number of students. $sort orders the summary according to average marks.

17. AGGREGATION FOR HIGHEST MARKS

Another aggregation can be used to find the highest marks obtained by students in each department.

```
db.students.aggregate([
  {$group: {
    _id: "$department",
   highestMarks: {$max: "$marks"}
  }},
  {$sort: {highestMarks: -1}}
])
```

This produces a department-wise summary in which highestMarks contains the maximum marks found in each department.

18. INDEX CREATION

An index improves the efficiency of queries on frequently searched fields. An index was created on rollNo because it is used as a student identifier.

```
db.students.createIndex({rollNo: 1})
```

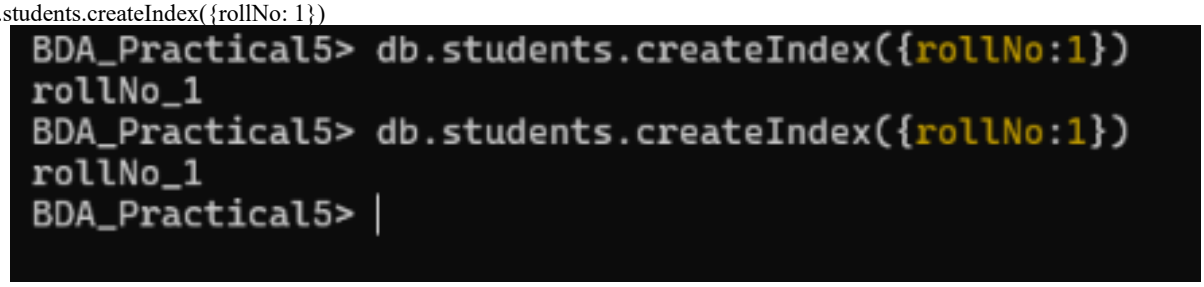


Figure 12: Creation and verification of the rollNo index.

```
db.students.getIndexes()
```

The query planner can be inspected using explain() and executionStats. This allows query execution information to be compared before and after indexing.

```
db.students.find({rollNo: 104}).explain("executionStats").queryPlanner.winningPlan
```

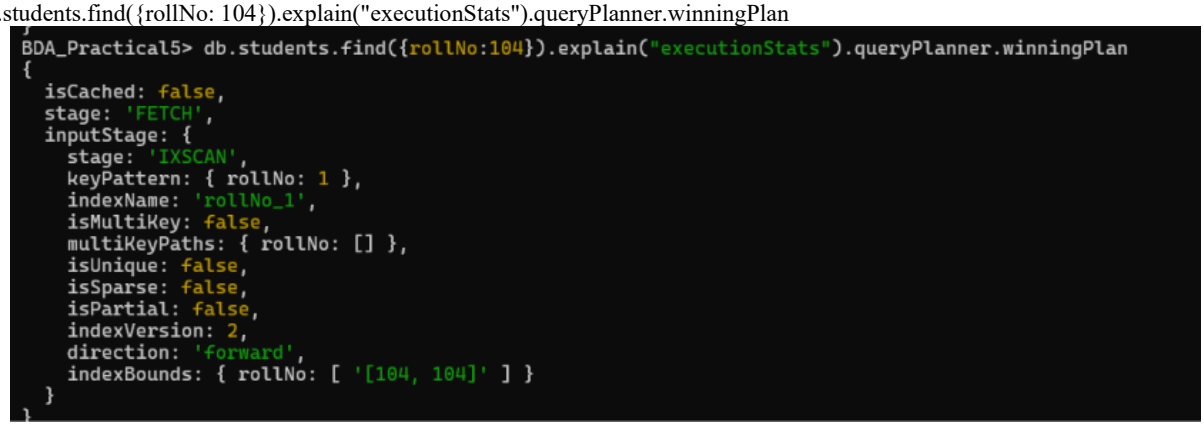


Figure 13: Query execution plan showing IXSCAN using rollNo_1.

The execution plan showed an IXSCAN stage using the rollNo_1 index. For a small collection, an index may not produce a large visible timing difference, but the query planner confirms that the index is being used.

19. JSON DATASET PREPARATION

A JSON dataset named students_dataset.json was prepared in the Practical 5 working folder. The file contains student records in JSON array format and can be imported into a separate MongoDB collection using mongoimport.

```
[
  {"rollNo":201,"name":"Meera","department":"IT","semester":5,"marks":84},
  {"rollNo":202,"name":"Jay","department":"CE","semester":5,"marks":79},
  {"rollNo":203,"name":"Nisha","department":"IT","semester":5,"marks":93},
  {"rollNo":204,"name":"Rahul","department":"CSE","semester":5,"marks":71},
  {"rollNo":205,"name":"Pooja","department":"IT","semester":5,"marks":87}
]
```

The JSON import step is treated as supplementary in this report because the submitted execution screenshots do not contain a successful mongoimport result.

```
mongoimport --uri="mongodb://127.0.0.1:27017/BDA_Practical5" --collection=importedStudents --file="students_dataset.json" --jsonArray
```

After import, the collection can be checked from mongosh.

```
use BDA_Practical5
db.importedStudents.find()
```

20. ANALYTICAL QUERIES ON IMPORTED DATA

The following queries can be used after the JSON dataset is imported into importedStudents.

```
db.importedStudents.find({marks: {$gte: 80}})
db.importedStudents.countDocuments({department: "IT"})
db.importedStudents.aggregate([
  {$group: {
    _id: "$department",
    totalStudents: {$sum: 1},
    averageMarks: {$avg: "$marks"},
    maximumMarks: {$max: "$marks"}
  }},
  {$sort: {averageMarks: -1}}
])
```

These queries provide a department-wise summary after the JSON data has been imported.

21. KEY QUESTIONS / ANALYSIS

Q1. What is MongoDB and how does it differ from relational databases?

MongoDB is a document-oriented NoSQL database. It stores data as BSON documents inside collections. Relational databases store data mainly in tables with rows and columns and normally use a predefined relational schema. MongoDB provides a flexible document structure, while relational systems commonly use structured schemas and relationships between tables.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for Create, Read, Update and Delete. These four operations represent the basic actions required to add data, retrieve data, modify existing data and remove unwanted data. MongoDB provides methods such as insertOne(), find(), updateOne() and deleteOne() for these operations.

Q3. How does MongoDB store data in BSON format?

MongoDB stores documents internally in BSON, or Binary JSON. BSON represents JSON-like documents in a binary form and supports several data types. This allows MongoDB to store semi-structured documents efficiently while maintaining a flexible document model.

22. KEY QUESTIONS / ANALYSIS - CONTINUED

Q4. What is the aggregation pipeline in MongoDB?

The aggregation pipeline is a sequence of processing stages used to transform and analyse documents. Common stages include $match for filtering, $group for grouping and calculations, $sort for ordering and $project for selecting or reshaping fields. The output of one stage becomes the input to the next stage.

Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases can be considered when the application handles large volumes of semi-structured or changing data, requires a flexible schema, or needs to scale data storage and processing across distributed systems. SQL databases remain suitable when structured schemas, complex relationships, strong relational constraints and transaction requirements are central to the application. The choice depends on the data and application requirements.

23. SUPPLEMENTARY PROBLEMS

23.1 Compare Query Performance With and Without an Index

Run a query on a sufficiently large collection before creating an index and record the execution statistics. Then create an index on the searched field and run the same query again. Compare executionStats, especially the number of documents examined and the execution time. On a small dataset, the difference may not be clearly visible.

```
db.students.find({rollNo: 103}).explain("executionStats")
db.students.createIndex({rollNo: 1})
db.students.find({rollNo: 103}).explain("executionStats")
```

23.2 Generate a Summary Report

Use aggregation to generate a department-wise report containing total students, average marks and maximum marks.

```
db.students.aggregate([
  {$group: {
    _id: "$department",
    totalStudents: {$sum: 1},
```

```
      averageMarks: {$avg: "$marks"},
      maximumMarks: {$max: "$marks"}
    }}
  ])
```

23.3 Import a JSON Dataset and Perform Analytical Queries

Create a JSON file containing student records, import it with mongoimport and use find() and aggregate() to answer questions such as how many students belong to a department, what the average marks are and what the maximum marks are.

24. REAL-WORLD USE OF MONGODB IN BIG DATA

MongoDB is used for applications that handle large and changing document-based data. Examples include content management, product catalogues, event and log data, user profiles, mobile and web applications and systems that receive data in different document structures. Its document model can reduce the need to split related application data across many tables.

25. MONGODB ATLAS

MongoDB Atlas is the cloud-hosted MongoDB service provided by MongoDB. It allows databases to be deployed and managed in the cloud without maintaining the database server infrastructure manually. Atlas can be used when an application requires a managed MongoDB deployment, remote access, monitoring and scalable cloud resources.

26. MONGODB AND MYSQL

MongoDB and MySQL use different data models. MongoDB uses BSON documents and collections, whereas MySQL uses relational tables and SQL. Performance depends on the workload, schema, indexes, hardware, query design and deployment architecture. Therefore, a meaningful comparison for large datasets should use the same hardware, dataset size and equivalent operations, and should measure execution time and resource usage rather than assuming that one database is always faster.

27. ADVANTAGES OF MONGODB

- Flexible document structure for semi-structured data.
- Documents can contain nested fields and arrays.
- Supports indexing for frequently queried fields.
- Aggregation pipeline provides built-in data processing and analysis.
- Suitable for applications where the data structure changes over time.
- Can be used in distributed and cloud-based deployments.

28. POST LABORATORY WORK

The following activities can be studied after completing the practical:

- Research real-world Big Data applications that use MongoDB.
- Compare MongoDB and MySQL using a large common dataset and equivalent queries.
- Study MongoDB Atlas and understand how a cloud-hosted MongoDB deployment is created and managed.
- Study how indexing changes query execution for large collections.

29. FINAL VERIFICATION

The practical requirements were verified through the following operations:

- MongoDB server and mongosh connection verified.
- BDA_Practical5 database and students collection created.
- Single and multiple document insertion performed using insertOne() and insertMany().
- Documents retrieved using find(), findOne(), filters, projection, sorting and limit().
- Existing documents modified using updateOne() and updateMany().
- Documents removed using deleteOne() and deleteMany(), and collection deletion demonstrated using drop().
- Aggregation pipeline used with $match, $group and $sort for data analysis.
- Index created on rollNo and query execution inspected using explain("executionStats").
- The JSON dataset file was prepared for import; a successful mongoimport result was not included in the submitted screenshots.
- CRUD, aggregation and indexing operations were verified through the submitted MongoDB shell screenshots.

30. CONCLUSION

In this practical, MongoDB was used as a NoSQL document database for managing student records. The database and collection were created and the basic CRUD operations were performed using MongoDB Shell. Documents were queried with filters, projection and sorting, updated and deleted. BSON, the aggregation pipeline and indexing were also studied. The practical demonstrates how MongoDB can be used to manage flexible semi-structured data in a Big Data environment.

31. FINAL RESULT

CRUD operations using MongoDB were implemented and studied successfully. Database and collection creation, document insertion, querying, updating, deletion, aggregation and indexing were verified using MongoDB Shell. A JSON dataset was also prepared as supplementary input for data import and analysis.